UniStays App Summary

What it is

UniStays is a React single-page application for university housing that connects students looking for accommodation with hosts who publish and manage listings, supported by analytics and contextual chat.

Who it is for

Primary persona: university students searching for housing near universities.

What it does

- Search listings by city/university with filters and a synchronized map view.
- Show rich listing pages with photos, amenities, reviews, and booking actions.
- Guide hosts through step-by-step listing creation with validation and pre-review.
- Provide host dashboards for listing management and performance analytics.
- Enable contextual chat tied to a specific listing and booking flow.
- Offer admin tools to review listings, manage cities/universities, and handle reports.

How it works (architecture)

- Frontend: React + Vite SPA with React Router and Redux; UI in src/ui and shared layers in src/core, src/application, src/infrastructure, src/shared.
- Auth: Clerk handles login; a Firebase Cloud Function verifies Clerk JWTs and issues Firebase custom tokens with optional admin claims.
- Data: Firestore stores users, apartments, conversations, reports, and reviews; Firebase Storage holds listing images; rules in firestore.rules.
- Maps and analytics: Google Maps JS API for maps/geocoding; Google Analytics (GA4) and Firebase Analytics via env configuration.
- Business logic is mostly client-side with security enforced by Firestore rules.

How to run (minimal)

- Install Node.js 20 and npm (per docs/01-setup.md).
- Run npm install in the repo root.
- Create .env with required VITE_* keys (Clerk, Firebase, Google Maps, EmailJS, GA) as documented in docs/01-setup.md.
- Start the dev server with npm run dev (Vite).

Key Flows (Repo Evidence)

Publish listing (Host)

- Host completes a step-by-step form with validation and help.
- Client geocodes the address, uploads images to Storage, and writes listing data to Firestore.
- Listing is set to pending_review until an admin verifies and publishes it.

Search and map sync (Student)

- Student opens the city route and filters published listings.
- Firestore query returns results for list and map in parallel.
- Hovering a list card highlights the corresponding map marker.

Booking to chat (Student -> Host)

- Booking form creates a payload and redirects to chat.
- Chat bootstrap starts a conversation if missing and posts a booking preview message.
- Conversation is stored in Firestore and linked to the listing.

Admin verification

- Admin panel lists pending_review listings.
- Admin updates status and publishedAt in Firestore.
- Platform message notifies the host of verification.

Data model hints

- Firestore collections include users, apartments, conversations, reports, reviews, and analytics subcollections.
- Listing images are stored in Firebase Storage paths like immagini/apt_<id> and immagini/apt_<id>/rooms/<roomId>.

Repo Map and Stack

Repository map

- src/: app bootstrap, routing, core/application/infrastructure layers, UI components, shared types.
- functions/: Firebase Cloud Functions for Clerk -> Firebase auth bridge.
- public/: static assets and SPA redirects.
- docs/: project documentation and architecture notes.
- firebase.json and firestore.rules: Firebase configuration and security rules.

Stack and services

- Frontend: React 18, Vite, React Router, Redux Toolkit, Tailwind CSS.
- Auth: Clerk with Firebase Auth custom tokens.
- Backend services: Firestore, Storage, Cloud Functions (region europe-west1).
- Integrations: Google Maps JS API, EmailJS, GA4, Firebase Analytics.

Build and deploy

- Frontend build: npm run build; preview via npm run preview.
- Deploy frontend with npm run deploy (Netlify CLI, dist/).
- Deploy functions: cd functions, npm install, set secrets, firebase deploy --only functions.